Unit III

3. Using Functions , Class- Objects, Forms in PHP

3.1 Functions in PHP: Defining and calling a function, Returning Value from function
3.2 Date and Time in PHP: Date and Time Functions
3.3 Strings in PHP: String Functions
3.4 Objects in PHP: What is Class & Object, Creating and accessing a Class & Object
3.5 Object properties, object methods, Overloading, inheritance, Constructor and Destructor
3.6 Creating HTML Form
3.7 Handling HTML Form data in PHP

3.1 Functions in PHP: Defining and calling a function, Returning Value from function:

Functions in PHP:

A **function** in PHP is a block of code that performs a specific task. It helps in **code reusability** and **modular programming**.

1. Defining a Function:

Use the `function` keyword to define a function.

Syntax:

```
php
    function functionName() {
        // code to execute
    }
```

Example:

```
php
    function greet() {
        echo "Hello, welcome!";
    }
```

2. Calling a Function:

Once defined, you can **call** the function by using its name followed by `()`.

Example:

```
php
    greet();  // Output: Hello, welcome!
```

3. Returning a Value from a Function:

A function can return a value using the `return` statement.

Example:

```
php
    function add($a, $b) {
        return $a + $b;
    }
    $result = add(5, 3);
```

```
echo $result;  // Output: 8
```

You can store the return value in a variable or use it directly.

Why Use Functions?

Reduces code repetition
Improves readability
Makes debugging easier

3.2 Date and Time in PHP: Date and Time Functions:

Date and Time in PHP:

PHP provides **built-in functions** to work with **date and time**, such as getting the current date, formatting it, or calculating time differences.

1. `date()` Function:

Used to **format** the current date and time.
Accepts format characters like `d` (day), `m` (month), `Y` (year), `H` (hour), `i` (minutes), etc.

Example:

php

```
echo date("d-m-Y");  // Output: 20-04-2025
echo date("h:i A");  // Output: 11:45 AM
```

2. `time()` Function:

Returns the **current Unix timestamp** (number of seconds since Jan 1, 1970).

Example:

php

```
echo time(); // Output: 1713604800 (example)
```

3. `mktime()` Function:

Creates a timestamp for a specific **date and time**.

php

```
$timestamp = mktime(0, 0, 0, 12, 25, 2025);
echo date("d-m-Y", $timestamp); // Output: 25-12-2025
```

4. `strtotime()` Function:

Converts a **human-readable date** into a timestamp.

php

```
$date = strtotime("next Friday");
echo date("d-m-Y", $date);
```

Why Use Date & Time Functions?

Display current time or date
Store timestamps
Schedule events or deadlines

Work with time zones

3.3 Strings in PHP: String Functions:

Strings in PHP:

A **string** is a sequence of characters used to store and manipulate text. PHP provides many **built-in string functions** to handle strings easily.

Common String Functions in PHP:

1. **`strlen()`** – Get String Length

   Returns the number of characters in a string.

   php

   ```
   $str = "Hello";
   echo strlen($str); // Output: 5
   ```

2. **`strtoupper()` and `strtolower()`** – Change Case

   Convert strings to upper or lower case.

   php

   ```
   echo strtoupper("hello"); // Output: HELLO
   echo strtolower("WORLD"); // Output: world
   ```

3. **`strpos()`** – Find Position of Substring

   Returns the position of the first occurrence of a substring.

   php

   ```
   echo strpos("Hello World", "World"); // Output: 6
   ```

4. **`str_replace()`** – Replace Text

   Replaces all occurrences of a word or character in a string.

   php

   ```
   echo str_replace("World", "PHP", "Hello World"); // Output: Hello PHP
   ```

5. **`substr()`** – Extract Part of a String

   Returns a part of a string.

   php

   ```
   echo substr("Hello PHP", 6); // Output: PHP
   ```

Why Use String Functions?

Useful for formatting and validating input
Helpful in search, replace, and text manipulation
Makes working with text easier and faster

3.4 Objects in PHP: What is Class & Object, Creating and accessing a Class & Object:

Objects in PHP:

PHP supports **Object-Oriented Programming (OOP)** using **classes** and **objects**. This approach helps organize code using real-world concepts.

1. What is a Class?

A **class** is a **template or blueprint** for creating objects.
It defines **properties (variables)** and **methods (functions)**.

```
php
    class Car {
        public $brand = "Toyota";

        public function start() {
            echo "Car is starting...";
        }
    }
```

2. What is an Object?

An **object** is an **instance of a class**.
It can **access properties and methods** of the class.

3. Creating a Class & Object:

Example:

```
    php
        // Define a class
        class Person {
            public $name;

            public function sayHello() {
                echo "Hello, my name is " . $this->name;
            }
        }

        // Create object
        $person1 = new Person();

        // Access property
        $person1->name = "Ayesha";

        // Call method
        $person1->sayHello(); // Output: Hello, my name is Ayesha
```

Key Terms:

Term	Description
`class`	Defines the structure (blueprint)
`object`	Instance of the class
`$this`	Refers to the current object
`->`	Used to access object members

Benefits of OOP in PHP:

Reusability of code
Better organization
Easier maintenance

3.5 Object properties, object methods, Overloading, inheritance, Constructor and Destructor:

1. Object Properties:

Variables defined inside a class.
Represent **characteristics** of an object.

php

```
class Car {
    public $brand = "Toyota";
}
```

2. Object Methods:

Functions defined inside a class.
Represent **actions** of an object.

php

```
class Car {
    public function start() {
        echo "Car started!";
    }
}
```

3. Constructor:

A special method that runs **automatically** when an object is created.
Declared using `__construct()`.

php

```
class Student {
    public $name;
    function __construct($n) {
        $this->name = $n;
    }
}
```

4. Destructor:

Runs **automatically** when an object is destroyed.
Declared using `__destruct()`.

php

```
function __destruct() {
```

```
        echo "Object destroyed.";
    }
```

5. Inheritance:

A class can **inherit** properties and methods from another class using the `extends` keyword.

```
php
    class Animal {
        public function sound() {
            echo "Animal sound";
        }
    }

    class Dog extends Animal {
        public function bark() {
            echo "Barking";
        }
    }
```

6. Overloading (Magic Methods):

Allows dynamic handling of properties or methods using **magic methods** like `__get()`, `__set()`, `__call()`.

```
php
    function __get($name) {
        echo "Trying to access $name";
    }
```

Summary Table:

Concept	Purpose
Properties	Store object data (variables)
Methods	Perform actions (functions)
Constructor	Auto-runs when object is created
Destructor	Auto-runs when object is destroyed
Inheritance	Reuse code from another class
Overloading	Handle undefined properties/methods

3.6 Creating HTML Form:

Creating HTML Form:

An **HTML form** is used to **collect user input** (like name, email, password) and send it to a server (often handled by PHP).

1. Basic Syntax of a Form:

```
html
```

```
<form action="submit.php" method="post">
    <!-- Form elements go here -->
</form>
```

`action`: The page where form data is sent.
`method`: How data is sent (`GET` or `POST`).

2. Common Form Elements:

html

```
<form action="submit.php" method="post">
    Name: <input type="text" name="username"><br>
    Email: <input type="email" name="email"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Submit">
</form>
```

`<input>`: Used for text, email, password, etc.
`name`: Important for accessing data in PHP.

3. `GET` vs `POST` Methods:

Method	Description
`GET`	Sends data via URL, visible
`POST`	Sends data securely, not visible

Purpose of HTML Forms:

Login pages
Signup forms
Contact forms
Surveys, feedback, etc.

Output Example:

A form on a web page where users can **enter details and submit** them to a PHP file.

3.7 Handling HTML Form data in PHP:

Handling HTML Form Data in PHP:

When a user submits a form, PHP can **collect and process** that data using `$_GET` or `$_POST` superglobals depending on the form method.

1. Create HTML Form:

html

```
<form action="formhandler.php" method="post">
    Name: <input type="text" name="username"><br>
    <input type="submit" value="Submit">
</form>
```

`action="formhandler.php"` – data is sent to this PHP file.
`method="post"` – data is sent using POST method.

2. Handle Form Data in PHP:

In `formhandler.php`:

php

```
<?php
    $name = $_POST['username'];
    echo "Hello, " . $name;
?>
```

`$_POST['username']` – fetches the data entered in the form.
Use `$_GET[]` if method is `get`.

3. Using isset() to Avoid Errors:

php

```
if (isset($_POST['username'])) {
    $name = $_POST['username'];
    echo "Welcome, $name";
}
```

Prevents errors if the form is not submitted yet.

4. Security Tip – Use `htmlspecialchars()`:

php

```
$name = htmlspecialchars($_POST['username']);
```

Protects from XSS (Cross Site Scripting).

Summary:

Step	Description
Create HTML Form	With input fields & submit button
Use `$_POST` or `$_GET`	To access form data in PHP
Sanitize Input	For security using `htmlspecialchars()`